FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos II

Unidad 1 / Semana 3

Año: 2025

Contenidos

Colas de planificación. Creación y terminación de procesos. Cambio de contexto. Hilos: tipos y componentes. Concepto de núcleos e hilos en procesadores.

Índice

1. Colas de Planificación 3
2. Creación y terminación de procesos 4
3. Cambio de Contexto / Estado de proceso 6
4. Hilos 6
4.1 Sistemas con soporte Monohilo / Multihilo 8
4.1.1 Sistemas sin soporte a Hilos - Monohilo 9
4.1.2 Sistemas con soporte a Hilos - Multihilos 10
4.2 Estados de los Hilos 12
4.3 Categorías de Hilos 15
4.3.1 Hilos de nivel de usuario (ULT) 15
4.3.2 Hilos de nivel de núcleo / kernel (KLT) 16
4.3.3 Enfoques combinados (ULT/KLT) 17
4.3.4 Relación Hilos / Procesos 17
5. Procesadores - Núcleos e hilos 18
5.1 ¿Qué es un procesador multinúcleo? 19
5.2 Funcionamiento de un procesador multinúcleo 19
5.3 Ventajas de un procesador multinúcleo 19
5.4 Arquitecturas y tipos de procesadores 20
5.4.1 Arquitecturas x86 vs. ARM 20
5.4.2 Tipos de procesadores por marca 20

1. Colas de Planificación

Se ha visto que los procesos tienen asociado un estado, dentro de la BCP o PCB que indica en qué situación está el proceso en un momento determinado.

Para administrar a los procesos con sus correspondientes estados, se usan unas estructuras de datos llamadas *colas*, las cuales son listas enlazadas con punteros a la PCB de cada proceso que tenga el estado que identifica a la cola..
Por ejemplo, en la cola de procesos *No Ejecutando* del modelo de dos estados se almacenarán todos los procesos esperando ser ejecutados.

De igual forma sucede con los procesos con estado nuevos, listos, bloqueados, etc del modelo de cinco estados, estarán en la cola que identifique al estado mencionado.

Modelo de proceso de dos estados

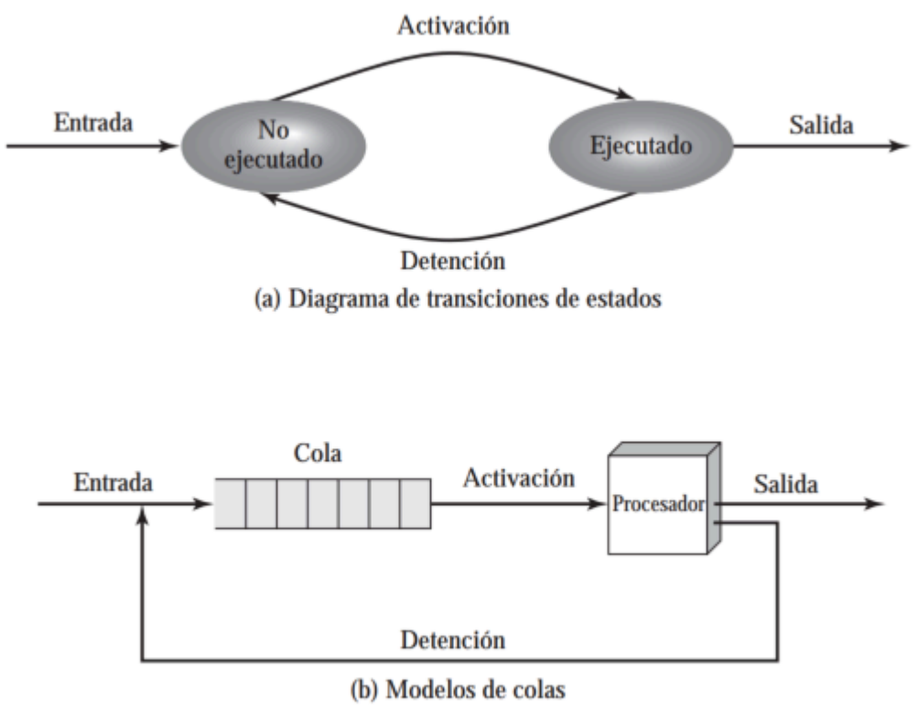


Figura 3.5. Modelo de proceso de dos estados.

Figura tomada de la lectura S2-L1

Modelo de proceso de cinco estados

En este modelo se agregan nuevos estados (ej. nuevo, listos, bloqueados, etc) que identifican con más detalle la situación del proceso dentro del sistema.

Por lo tanto, se crearán colas de planificación correspondientes a los estados que componen este modelo, para almacenar los procesos que tengan el estado que identifica a la cola.

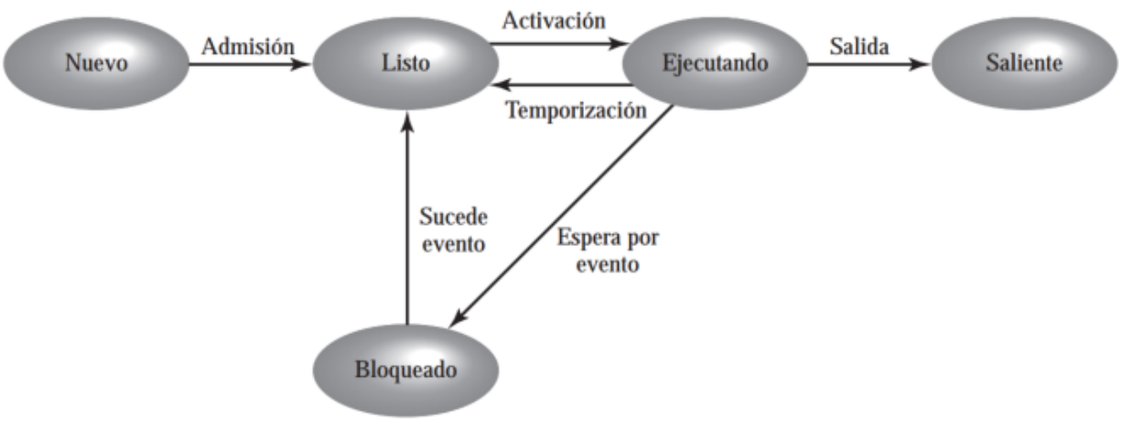


Figura tomada de la lectura S2-L1

Estos temas están incluídos en la lectura *S2-L1*, por ese motivo no profundizaremos estos conceptos aquí.

2. Creación y terminación de procesos

TABLA 3.1. Razones para la creación de un proceso.

Nuevo proceso de lotes	El sistema operativo dispone de un flujo de control de lotes de trabajos, habitualmente una cinta un disco. Cuando el sistema operativo está listo para procesar un nuevo trabajo, leerá la siguiente secuencia de mandatos de control de trabajos.
Sesión interactiva	Un usuario desde un terminal entra en el sistema.
Creado por el sistema operativo para proporcionar un servicio	El sistema operativo puede crear un proceso para realizar una función en representación de un programa de usuario, sin que el usuario tenga que esperar (por ejemplo, un proceso para controlar la impresión).
Creado por un proceso existente	Por motivos de modularidad o para explotar el paralelismo, un programa de usuario puede ordenar la creación de un número de procesos.

Tabla 3.2. Razones para la terminación de un proceso.

Finalización normal	El proceso ejecuta una llamada al sistema operativo para indicar que ha completado su ejecución
Límite de tiempo excedido	El proceso ha ejecutado más tiempo del especificado en un límite máximo. Existen varias posibilidades para medir dicho tiempo. Estas incluyen el tiempo total utilizado, el tiempo utilizado únicamente en ejecución, y, en el caso de procesos interactivos, la cantidad de tiempo desde que el usuario realizó la última entrada.
Memoria no disponible	El proceso requiere más memoria de la que el sistema puede proporcionar.
Violaciones de frontera	El proceso trata de acceder a una posición de memoria a la cual no tiene acceso permitido.
Error de protección	El proceso trata de usar un recurso, por ejemplo un fichero, al que no tiene permitido acceder, o trata de utilizarlo de una forma no apropia, por ejemplo, escribiendo en un fichero de sólo lectura.
Error aritmético	El proceso trata de realizar una operación de cálculo no permitida, tal como una división por 0, o trata de almacenar números mayores de los que la representación hardware puede codificar.
Límite de tiempo	El proceso ha esperado más tiempo que el especificado en un valor máximo para que se cumpla un determinado evento.
Fallo de E/S	Se ha producido un error durante una operación de entrada o salida, por ejemplo la imposibilidad de encontrar un fichero, fallo en la lectura o escritura después de un límite máximo de intentos (cuando, por ejemplo, se encuentra un área defectuosa en una cinta), o una operación inválida (la lectura de una impresora en línea).
Instrucción no válida	El proceso intenta ejecutar una instrucción inexistente (habitualmente el resultado de un salto a un área de datos y el intento de ejecutar dichos datos).
Instrucción privilegiada	El proceso intenta utilizar una instrucción reservada al sistema operativo.
Uso inapropiado de datos	Una porción de datos es de tipo erróneo o no se encuentra inicializada.
Intervención del operador por el sistema operativo	Por alguna razón, el operador o el sistema operativo ha finalizado el proceso (por ejemplo, se ha dado una condición de interbloqueo).
Terminación del proceso padre	Cuando un proceso padre termina, el sistema operativo puede automáticamente finalizar todos los procesos hijos descendientes de dicho padre.
Solicitud del proceso padre	Un proceso padre habitualmente tiene autoridad para finalizar sus propios procesos descendientes.

Las razones de finalización de un proceso se pueden resumir en cuatro categorías según A.Tanenbaum[1]:

1. - Salida normal (voluntaria).
   El programa finaliza su ejecución normalmente, sin errores.

2. - Salida por error (voluntaria).
   El mismo programa detecta un error y detiene su ejecución informando el error encontrado.

3. - Error fatal (involuntaria).
   Se produce un error fatal en el proceso, que el sistema operativo detecta y finaliza el proceso.

[1] Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

4. - Eliminado por otro proceso (involuntaria).
   En este caso, otro proceso, con los permisos necesarios solicita al sistema operativo la finalización del proceso.

Estos temas están incluídos en la lectura *S2-L1*, por ese motivo no profundizaremos estos conceptos aquí.

3. Cambio de Contexto / Estado de proceso

Esta tarea se debe realizar cuando el proceso en ejecución pasa a otro estado (Listo, Bloqueado, etc.). Requiere acciones más complejas por parte del sistema operativo:

Pasos principales:

- Guardar el estado del procesador (contador de programa, registros).
- Actualizar el bloque de control del proceso saliente, incluyendo su nuevo estado y motivos del cambio.
- Mover el bloque de control a la cola correspondiente (Listo, Bloqueado, etc.).
- Seleccionar un nuevo proceso para ejecutar.
- Actualizar el bloque de control del proceso entrante, marcándolo como Ejecutando.
- Actualizar estructuras de memoria, si es necesario.
- Restaurar el estado del procesador del nuevo proceso (contador de programa y registros).

Estos temas están incluídos en la lectura *S2-L1*, por ese motivo no profundizaremos estos conceptos aquí.

4. Hilos

Tradicionalmente, el concepto de proceso ha incluído dos características principales: la propiedad de recursos y la planificación/ejecución.

Propiedad de recursos

Un proceso incluye un espacio de direcciones virtuales para gestionar su imagen (programa, datos, pila y atributos del bloque de control del proceso). A un proceso se le pueden asignar recursos como memoria principal, canales y dispositivos de E/S, y archivos. El sistema operativo se encarga de proteger estos recursos para evitar interferencias entre procesos.

Planificación/ejecución

La ejecución de un proceso sigue una ruta de ejecución a través de uno o más programas, pudiendo intercalarse con otros procesos. Un proceso tiene un *estado* de ejecución (Ejecutando, Listo, Bloqueado, etc.) y una *prioridad*, siendo la entidad que el sistema operativo planifica y activa.

Sin embargo, en los sistemas operativos modernos, se ha reconocido que estas dos características son *independientes* y pueden tratarse por separado. Esta distinción ha llevado al desarrollo del concepto de **hilo (thread) o proceso ligero**.

Para diferenciar estas características:

• La unidad de propiedad de recursos se denomina ***proceso o tarea***.

• La unidad de ejecución que se activa y planifica se denomina ***hilo (thread) o proceso ligero[2].***

Los hilos pueden definirse entonces como ***unidades de ejecución que operan dentro de un mismo proceso, compartiendo su espacio de memoria y recursos***.

Por esta razón, suelen denominarse *procesos livianos* o *subprocesos*, ya que permiten realizar múltiples tareas de forma concurrente con menor sobrecarga que los procesos tradicionales.

Ejemplos de Aplicaciones y Usos de Hilos

- **Servidor de archivos:** Un servidor de archivos puede ejecutar un nuevo hilo para cada nueva petición, permitiendo la creación y finalización rápida de muchos hilos. En un sistema multiprocesador, múltiples hilos del mismo proceso pueden ejecutarse simultáneamente. Además, la coordinación es más rápida usando hilos y memoria compartida que procesos y paso de mensajes.
- **Trabajo en primer plano y en segundo plano:** En una *hoja de cálculo*, un hilo podría manejar la interfaz de usuario mientras otro ejecuta comandos y actualiza la hoja, mejorando la percepción de velocidad.
  En un *procesador de texto*, tareas como el guardado automático, repaginado del documento y captura de teclado / mouse pueden implementarse como hilos separados, simplificando el código principal y mejorando la experiencia de uso y el tiempo de respuesta.

[2] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

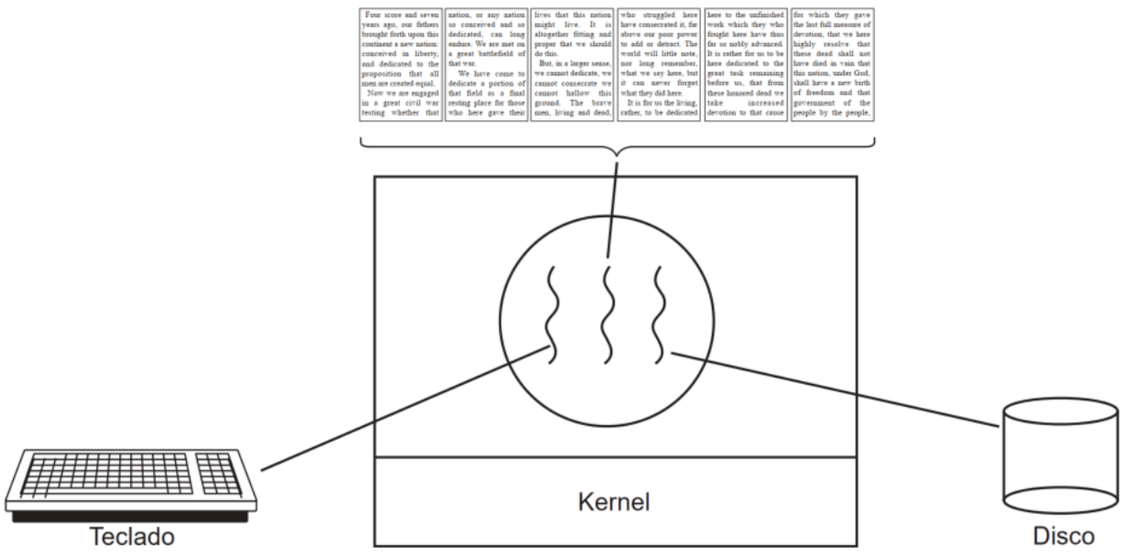


Figura 1. Procesador de texto con tres hilos, uno para atender teclado / mouse, otro para reformatear el documento y el tercero para guardar periódicamente los datos.[3]

4.1 Sistemas con soporte Monohilo / Multihilo

Para un sistema operativo moderno es muy importante contar con soporte a hilos, porque permite que un proceso se divida en *múltiples hilos de ejecución* que comparten recursos como la memoria. Esta arquitectura posibilita la ejecución paralela de tareas, lo que se traduce en una mejora significativa del rendimiento general y una mayor capacidad de respuesta del sistema.

[3] Tanenbaum, Andrews (2009), Sistemas Operativos Modernos. Tercera edición

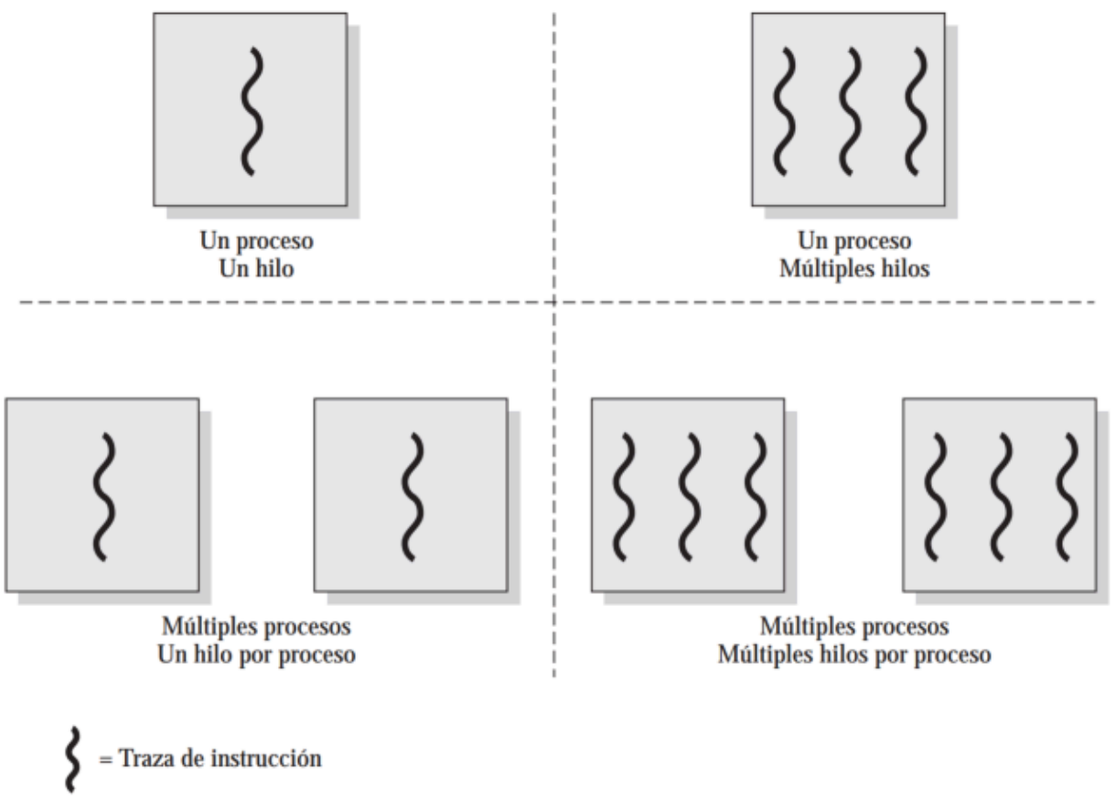


Figura 2. Izquierda: Sin soporte a HILOS. Derecha: Con soporte a HILOS[4]

4.1.1 Sistemas sin soporte a Hilos - Monohilo

Un sistema operativo tradicional, que no tiene soporte a hilos, es aquel que sólo contempla **una sola unidad de ejecución dentro de un proceso** y se denominan *monohilos*. No existe el concepto de hilo para ellos.

Al ejecutar una única secuencia de instrucciones por proceso, simplifican la planificación pero limitan la optimización del uso del hardware disponible principalmente en el multiprocesamiento.
En este caso no hay elementos nuevos dentro del bloque del proceso, se mantienen los elementos ya descritos anteriormente en relación a los recursos como espacio de direcciones de memoria, pila, datos del proceso almacenados en la PCB.

Ejemplo: MS-DOS

[4] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

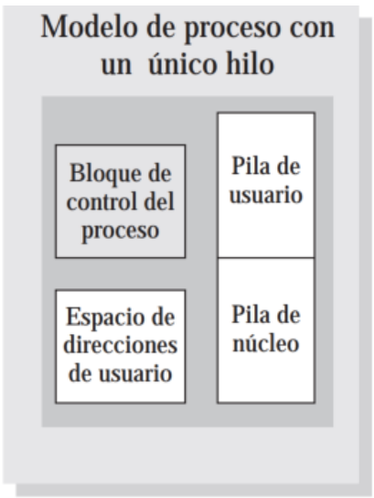


Figura 3. Sistema sin soporte a HILOS (Monohilo)[5]

4.1.2 Sistemas con soporte a Hilos - Multihilos

Los sistemas operativos modernos ya cuentan con el soporte a hilos. Esto significa que son capaces de administrar más de una unidad de ejecución dentro del mismo proceso (multihilo).

En un entorno multihilo:

Cada proceso tiene :

- Un espacio de direcciones virtuales que soporta la imagen del proceso.
- Acceso protegido al procesador, otros procesos (para comunicación), archivos y recursos de E/S. (Ya visto anteriormente en *procesos*)

Dentro de un proceso puede haber uno o más hilos, cada uno con:

- Un **estado de ejecución propio** (*Ejecutando, Listo, etc.*).
- Un **contexto de hilo** que se guarda cuando no está en ejecución (se puede ver como un *contador de programa* independiente para cada hilo dentro de un proceso).
- **Una pila de ejecución**.
- **Espacio de almacenamiento** para variables locales por cada hilo.
- **Acceso compartido a la memoria y recursos de su proceso**, con todos los demás hilos del mismo proceso.

En la figura 3 y 4 ilustran esta diferencia: en un modelo monohilo, el proceso incluye su bloque de control, espacio de direcciones y pilas de usuario/núcleo. En un modelo multihilo, el proceso sigue teniendo un único bloque de control y espacio de direcciones, pero hay múltiples pilas separadas y bloques de control para cada hilo, que contienen registros, prioridad y otra información de estado del hilo.

[5] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

Sistemas operativos como Windows o Linux son ejemplos de aquellos que soportan múltiples procesos, y cada uno de ellos soporta a su vez múltiples hilos.

Beneficios del Uso de Hilos

Los mayores beneficios de los hilos se manifiestan en el rendimiento y la eficiencia:

1. **Menor tiempo de creación**: Crear un nuevo hilo en un proceso existente es mucho más rápido que crear un proceso completamente nuevo.
   Estudios comparativos mostraron que la creación de un hilo es decenas o cientos de veces más rápida que la de un proceso por las tareas adicionales que incluye para el proceso. El hilo se crea dentro de un proceso ya existente.
2. **Menor tiempo de finalización**: Finalizar un hilo toma menos tiempo que finalizar un proceso.
3. **Menor tiempo de cambio de contexto**: Cambiar entre dos hilos dentro del mismo proceso es más rápido que cambiar entre procesos.
4. **Comunicación eficiente**: Los hilos mejoran la eficiencia de la comunicación entre diferentes programas. Dado que los hilos dentro de un mismo proceso comparten memoria y archivos, pueden comunicarse entre sí sin necesidad de invocar al núcleo, a diferencia de la comunicación entre procesos independientes que a menudo requiere la intervención del kernel del sistema.
5. **Estructura de programas simplificada**: Son útiles para simplificar la estructura de programas que realizan varias funciones diferentes, incluso en un solo procesador.

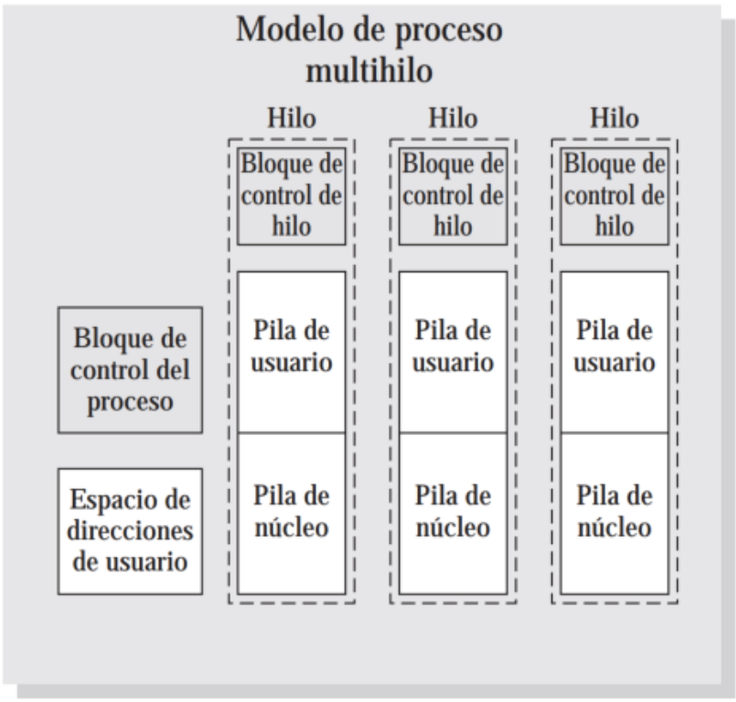


Figura 4. Sistema con soporte a HILOS (Multihilo)[6]

4.2 Estados de los Hilos

Los hilos, al igual que los procesos, tienen estados de ejecución y son independientes de los estados del proceso.

Los principales estados de los hilos son:

- **Ejecutando**: El hilo está utilizando el procesador.
- **Listo**: El hilo está esperando ser ejecutado por el procesador.
- **Bloqueado**: El hilo está esperando por un evento (ej. una operación de E/S).

Los estados de **suspensión** son conceptos a *nivel de proceso* y no se aplican directamente a los hilos, ya que si un proceso es expulsado de la memoria, todos sus hilos se suspenden al compartir el mismo espacio de direcciones.

Operaciones básicas

Las operaciones básicas realizadas con hilos que generen un cambio de estado son:

- **Creación**: Cuando se crea un nuevo proceso, se crea un hilo para dicho proceso. Posteriormente, un hilo puede crear otro hilo dentro del mismo

[6] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

proceso, asignándole su propio contexto de registro, espacio de pila y colocándolo en la cola de ***Listos***.

- **Bloqueo**: Cuando un hilo necesita esperar por un evento, se bloquea, guardando sus registros de usuario, contador de programa y punteros de pila. El procesador puede entonces ejecutar otro hilo, ejecutando un cambio de contexto y se guarda en la cola de ***bloqueados***.

- **Desbloqueo**: Cuando el evento por el que un hilo estaba bloqueado ocurre, el hilo pasa a la cola de ***Listos***.

- **Finalización**: Cuando un hilo completa su tarea, se liberan su registro de contexto y pilas.

Un aspecto crítico es si el bloqueo de un hilo implica el bloqueo de todo el proceso. Si un hilo bloqueado bloquea al proceso completo, se pierde parte de la potencia y flexibilidad de los hilos. Esta es una distinción importante entre los hilos a nivel de usuario y a nivel de núcleo.

Sincronización de Hilos

Debido a que todos los hilos de un proceso *comparten el mismo espacio de direcciones y recursos* (como archivos abiertos), es esencial sincronizar sus actividades. Cualquier alteración de un recurso por un hilo afecta a todos los demás hilos del mismo proceso. La sincronización previene interferencias y corrupción de estructuras de datos (ej. al añadir elementos a una lista enlazada). Las técnicas de sincronización para hilos son similares a las utilizadas para procesos y serán estudiadas más adelante.

Ejemplo - Google Chrome

El navegador Chrome emplea un modelo de **multiproceso e hilos** para mejorar rendimiento y estabilidad. Algunos de los hilos más importantes:

Hilo	Función principal
Hilo de interfaz (UI)	Maneja la interacción del usuario, eventos de teclado, mouse, etc.
Hilo de renderizado	Dibuja el contenido de la página web (HTML, CSS, JS) en la pantalla.
Hilo de red	Gestiona las solicitudes HTTP/HTTPS, descargas, y conexiones.
Hilo de JavaScript	Ejecuta el código JS en el motor V8.
Hilos de extensiones	Ejecutan scripts y procesos de extensiones instaladas.
Hilos de audio/video	Procesan medios en tiempo real, como reproducción de YouTube o videollamadas.

Chrome también lanza procesos separados por pestaña, pero dentro de cada proceso hay múltiples hilos trabajando en paralelo.

¿Qué pasa si el sistema no soporta hilos?

Si el sistema operativo no tiene soporte para hilos, podrían ocurrir varias limitaciones:

- No hay ejecución paralela real: todo se ejecuta de forma secuencial, incluso si el lenguaje lo permite.

- Rendimiento degradado: tareas como cargar páginas, reproducir medios o responder al usuario se vuelven muy lentas.

4.3 Categorías de Hilos

Existen tres categorías principales para la implementación de hilos:

1. Hilos de Nivel de Usuario (ULT - User-Level Threads)
2. Hilos de Nivel de Núcleo (KLT - Kernel-Level Threads)
3. Enfoques Combinados (ULT/KLT)

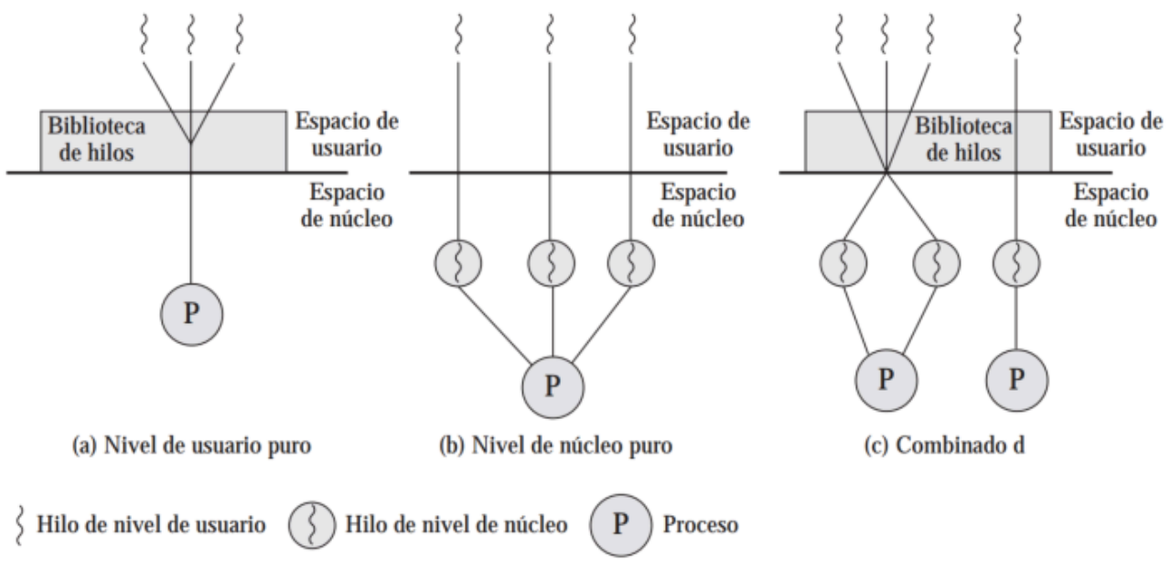


Figura 5 - Hilos de nivel de Usuario (ULT) y de núcleo / kernel (KLT)[7]

4.3.1 Hilos de nivel de usuario (ULT)

- **Gestión:** Toda la gestión de hilos es realizada por la aplicación en el espacio de usuario, a través de una biblioteca de hilos.

- **Visibilidad para el núcleo:** El núcleo no es consciente de la existencia de los hilos; solo planifica el proceso como una unidad y le asigna un único estado.

- **Operaciones:** La creación, destrucción, paso de mensajes, planificación y guardado/restauración de contexto se realizan en el espacio de usuario, sin intervención del núcleo.
    - **Ventajas:**
        - Cambio de contexto rápido: No requiere privilegios de modo núcleo, lo que evita la sobrecarga de dos cambios de modo (usuario a núcleo y núcleo a usuario).

[7] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño,Quinta edición

- **Planificación personalizable:** La aplicación puede especificar su propio algoritmo de planificación para los hilos, sin afectar al planificador del sistema operativo.

- **Portabilidad:** Los ULT pueden ejecutarse en cualquier sistema operativo sin modificaciones del núcleo, ya que la biblioteca de hilos es un conjunto de utilidades a nivel de aplicación.

- **Desventajas:**
  - Bloqueo del proceso: Si un ULT realiza una llamada al sistema bloqueante, todos los hilos del proceso se bloquean, ya que el núcleo solo ve y gestiona el proceso como una entidad única.

  - **Sin multiprocesamiento real:** Una aplicación multihilo ULT no puede aprovechar múltiples procesadores, porque el núcleo asigna el proceso a un solo procesador a la vez, por lo que solo un hilo del proceso puede ejecutar simultáneamente.

- **Soluciones a las desventajas:**

  - Convertir una llamada al sistema bloqueante en no bloqueante mediante la técnica de "**jacketing**" (revestimiento), donde una rutina a nivel de aplicación verifica la disponibilidad del dispositivo de E/S antes de bloquearse y pasar el control a otro hilo.

  - Implementar la aplicación con múltiples procesos en lugar de múltiples hilos, aunque esto introduce una mayor sobrecarga.

4.3.2 Hilos de nivel de núcleo / kernel (KLT)

- **Gestión:** El núcleo es quien realiza toda la gestión de hilos.

- **Visibilidad para el núcleo:** El núcleo mantiene información de contexto para el proceso y para cada hilo individual del proceso.

- **Planificación:** La planificación es realizada por el núcleo a nivel de hilo.
  - **Ventajas:**
    - Multiprocesamiento real: El núcleo puede planificar simultáneamente múltiples hilos de un mismo proceso en múltiples procesadores.

- No bloquea el proceso completo: Si un hilo de un proceso se bloquea, el núcleo puede planificar otro hilo del mismo proceso para su ejecución.
- Las rutinas del núcleo pueden ser multihilo.

- **Desventajas:**
  - Mayor sobrecarga: La transferencia de control de un hilo a otro del mismo proceso requiere un cambio de modo al núcleo, lo que genera una sobrecarga mayor que en los ULT.

- **Ejemplo:** Windows / Linux son ejemplos de este enfoque.

4.3.3 Enfoques combinados (ULT/KLT)

Algunos sistemas operativos ofrecen utilidades combinadas, siendo **Solaris** un ejemplo destacable.

- **Gestión:** La creación de hilos, la mayor parte de la planificación y la sincronización se realizan en el espacio de usuario.
- **Relación:** Múltiples ULT de una aplicación se asocian a un número menor o igual de KLT.
- **Ventajas:** Este enfoque busca combinar las ventajas de ambos enfoques puros (ULT y KLT), permitiendo la ejecución paralela en múltiples procesadores y evitando que una llamada al sistema bloqueante afecte al proceso completo, al tiempo que minimiza las desventajas.

4.3.4 Relación Hilos / Procesos

Además de la relación **muchos-a-uno** (**M:1**) de hilos a procesos (la base del multihilo), se han explorado otras configuraciones:

- **1:1 (Hilo:Proceso):** Cada hilo de ejecución es un único proceso con su propio espacio de direcciones y recursos. Es el modelo de las implementaciones UNIX tradicionales (Monohilo).

- **M:1 (Hilos:Proceso):** Un proceso define un espacio de direcciones y recursos, y múltiples hilos pueden crearse y ejecutarse dentro de ese proceso. Ejemplos: Windows NT, Solaris, Linux, OS/2, MACH.(Multihilo)

- **1:M (Hilo:Procesos):** Un solo hilo puede migrar de un entorno de proceso a otro, incluso entre máquinas diferentes en sistemas distribuidos. Esto permite la fácil movilidad de hilos entre distintos sistemas.
  Ejemplos: Ra (Clouds), Emerald.
  Este modelo está motivado por la necesidad de herramientas de estructuración para el programador, donde un hilo puede moverse entre

distintos dominios (espacios de direcciones) para manejar, por ejemplo, subprogramas de E/S de manera más eficiente y sin la sobrecarga de creación de procesos.

- **M:N (Hilos:Procesos):** Combina atributos de M:1 y 1:M, permitiendo que múltiples hilos se muevan entre múltiples dominios (procesos).
  Explorada en el sistema operativo experimental TRIX. Un dominio en TRIX es una entidad estática con un espacio de direcciones y puertos, y un hilo es una ruta de ejecución con su propia pila y estado.

  Los últimos dos casos no tienen muchos casos de uso reales, sino más bien se hicieron con fines educativos y de experimentación.

Hilos:Procesos	Descripción	Sistemas de Ejemplo
1:1	Cada hilo de ejecución es un único proceso con su propio espacio de direcciones y recursos.	Implementaciones UNIX tradicionales.
M:1	Un proceso define un espacio de direcciones y pertenencia dinámica de recursos. Se pueden crear y ejecutar múltiples hilos en ese proceso.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH.
1:M	Un hilo puede migrar de un entorno de proceso a otro. Esto permite a los hilos moverse fácilmente entre distintos sistemas.	Ra (Clouds), Emerald.
M:N	Combina atributos de M:1 y casos 1:M.	TRIX.

Figura 6. Relación entre hilos y procesos[8]

5. Procesadores - Núcleos e hilos

En el hardware de un procesador, los núcleos y los hilos son elementos esenciales que determinan su rendimiento.



Núcleos (Cores): Son las unidades de procesamiento **físicas**. Un procesador con múltiples núcleos puede ejecutar varias tareas a la vez de forma independiente, lo que aumenta la capacidad de procesamiento general del sistema.

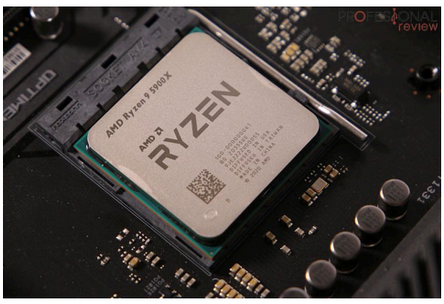
[8] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición.

Hilos (Threads): Los hilos pueden considerarse como “núcleos virtuales” generados mediante tecnologías como Hyper-Threading (Intel) o Simultaneous Multithreading (AMD). Aunque un núcleo representa una unidad física de procesamiento, los hilos permiten que este gestione múltiples tareas de forma más eficiente. Lo hacen alternando rápidamente entre procesos, lo que da la impresión de que se ejecutan simultáneamente. Este paralelismo mejora el aprovechamiento de los recursos del procesador y aumenta significativamente su rendimiento.

A continuación describiremos en forma más detallada los conceptos de núcleos e hilos en un procesador.

5.1 ¿Qué es un procesador multinúcleo?

Un procesador multinúcleo es un chip de circuito integrado que contiene múltiples unidades de procesamiento independientes, conocidas como núcleos. Cada núcleo puede ejecutar instrucciones por separado y de manera simultánea, lo que aumenta la potencia de procesamiento y mejora la capacidad de realizar varias tareas a la vez.



5.2 Funcionamiento de un procesador multinúcleo

Un procesador multinúcleo distribuye las tareas de cálculo entre sus diversos núcleos. Este procesamiento paralelo permite ejecutar las tareas de manera más rápida y eficiente, ya que cada núcleo puede manejar una parte independiente de la carga de trabajo. Sin embargo, para aprovechar al máximo las ventajas de un procesador multinúcleo, el sistema operativo y el software deben estar optimizados para utilizar todos los núcleos disponibles.

5.3 Ventajas de un procesador multinúcleo

El uso de un procesador multinúcleo ofrece varios beneficios:

- Mayor velocidad y capacidad de respuesta: Las tareas se pueden dividir y ejecutar en paralelo, lo que resulta en una experiencia informática más fluida y rápida.
- Multitarea mejorada: Permite ejecutar varias aplicaciones al mismo tiempo sin una degradación significativa del rendimiento.
- Mayor eficiencia energética: Los núcleos no utilizados pueden entrar en un estado de bajo consumo, lo que ayuda a reducir el consumo total de energía del sistema.

5.4 Arquitecturas y tipos de procesadores

El mercado de procesadores está dominado por dos grandes competidores: Intel y AMD. Se segmentan principalmente por la cantidad de núcleos y la arquitectura.

- CPU de un solo núcleo (Single-Core): Prácticamente obsoletos para el uso moderno, estos procesadores tienen una única unidad de ejecución. Son inadecuados para la multitarea.
- CPU multinúcleo (Multi-Core): Son el estándar actual. Contienen múltiples núcleos en un solo chip, lo que permite una verdadera multitarea y un rendimiento superior en aplicaciones optimizadas. Un procesador de 4 o 6 núcleos es ideal para la mayoría de las tareas de oficina.

5.4.1 Arquitecturas x86 vs. ARM

- x86: Es la arquitectura dominante en ordenadores de sobremesa y servidores (usada por Intel y AMD), conocida por su alto rendimiento y amplia compatibilidad con software.
- ARM: Destaca por su eficiencia energética y domina el mercado de dispositivos móviles y tabletas. Recientemente, ha ganado terreno en ordenadores portátiles.

5.4.2 Tipos de procesadores por marca

- Procesadores Intel Core: La línea más popular de Intel. Se dividen en gamas según el rendimiento:
    - Core i3: Tareas básicas.
    - Core i5: El estándar para la mayoría de las oficinas.
    - Core i7: Para usuarios avanzados y tareas exigentes como diseño o programación.
    - Core i9: Máximo rendimiento para estaciones de trabajo y profesionales.
- Procesadores AMD Ryzen: Competencia directa de Intel, AMD Ryzen ha ganado cuota de mercado por su excelente rendimiento multinúcleo y precios competitivos. Su segmentación es similar: Ryzen 3, 5, 7 y 9, ofreciendo una sólida alternativa en cada gama.



Lectura S2-L1

Esta semana incluye contenidos incluídos en la lectura S2-L1. Sugerimos una relectura del mismo para acompañar los temas aquí presentados



Actividad (S3-A1)

Realice la actividad S3-A1 publicada en el campus.